SCA101 - LAB 2_1A (EXPLORER)

Instruction Power Explorer

Step-by-step student guide: edit the numbers, watch the trace change

In Lab 2_1A you capture four power traces and compare them by eye. This guide goes with a **second, hands-on notebook** where you change the numbers yourself - zoom, smooth, count, measure and predict - and watch the plots update, so you can see exactly **why** a power trace looks the way it does. No hardware is needed for the first six steps.

> **HOW TO USE THIS GUIDE**
> This guide goes **with** the notebook - it does not replace it. Keep the notebook open next to this PDF. Every step says what to edit, what you should see, and what to try if you do not see it.

Which files are which

File (folder: jupyter/courses/sca101/)	What it is	Open it?
`Lab 2_1A - Instruction Power Explorer.ipynb`	The Explorer. Everything in this guide happens here.	**Yes**
`Lab 2_1A - Instruction Power Differences (MAIN).ipynb`	The original lab. Do it before or after the Explorer.	Optional
`... (HARDWARE).ipynb`, `... (SIMULATED).ipynb`	Back-end helpers. The notebooks run them for you.	**No**
`traces/lab2_1A_trace0.npy` to `trace3.npy`	The four recorded power traces the Explorer uses.	No (just keep them)

The four colored boxes you will see everywhere

DO THIS
An action to take.

YOU SHOULD SEE
What the screen should show.

IF IT DOESN'T
What to try if it didn't.

WATCH OUT
A mistake that costs time.

Guide version 1.0 - October 2026. Every number printed in this guide (pulse counts, cycle counts, correlations) was produced by running the notebook, headless and in a real Jupyter session, on **Ubuntu 22.04 (Python 3.10)**. Step 7 needs a ChipWhisperer and is described from the notebook's code; it has not yet been run on a live Husky for this guide.

Contents

The big picture . . . 3

Before you start . . . 4

How every edit-and-run cell works . . . 4

Setup - open the notebook and run it . . . 5

S.1 Open the Explorer . . . 5

S.2 Choose the version . . . 5

S.3 Run the two Setup cells . . . 5

S.4 Optional: run everything once . . . 5

Steps 1 to 3 - read a trace . . . 6

1 Look at a power trace . . . 6

2 How fast is the clock? (measure it yourself) . . . 7

3 Compare two traces . . . 8

Steps 4 and 5 - find the loop . . . 9

4 Find the loop . . . 9

5 Why does the divide loop speed up? . . . 11

Step 6 - what if the loop ran 5 times? Or 50? . . . 12

Step 7 - your turn on the real chip (hardware only) . . . 14

Step 8 - check your understanding . . . 15

You are done when... . . . 15

Troubleshooting . . . 16

The first six steps (no hardware) . . . 16

Step 7 (hardware) . . . 17

Appendix A - every variable you can edit . . . 18

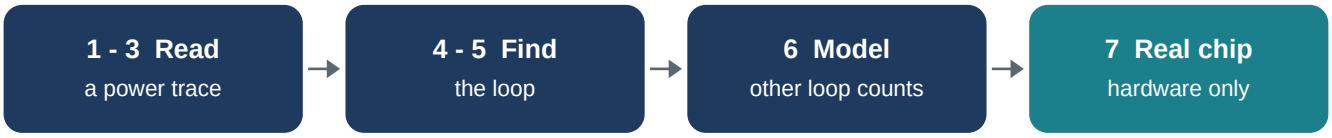
Appendix B - Glossary . . . 19

The big picture

What is this about? A microcontroller does its work one clock cycle at a time, and the electricity it draws changes a little with what it is doing. The ChipWhisperer records that as a **power trace**. If you know how to read the trace you can tell *what the chip is doing and how long it takes* - without ever seeing its code. That is the idea behind every side-channel attack in this course.

What is different about the Explorer? The original lab shows you four traces. The Explorer lets you **poke at them**: every code cell has a clearly marked block of numbers you can change. Change one, press **Shift+Enter**, and the plot and the printed numbers change straight away.

Your path through the notebook:



Step	What you do	What you learn	Hardware?
1	Zoom, smooth and flip a real trace.	What the x and y axes mean (and that the trace is upside down).	No
2	Measure the clock with autocorrelation.	There are **4 samples per clock cycle**.	No
3	Overlay two traces and slide one.	Single instructions are hard to see; changes shift later activity.	No
4	Find the loop's pulses automatically.	20 pulses; how smoothing changes the count; the divide loop's pulses move closer.	No
5	Compare the size of A with the time per trip.	**Data changes timing**: the divide gets faster as the number shrinks.	No
6	Stack real loop trips for any loop count.	How loop count changes the trace; when noise hides pulses.	No
7	Edit the C code on the real chip and capture.	Check your predictions on real silicon.	**Yes**
8	Answer six questions.	Put it all together.	No

Before you start

Tick every row. Most problems later come from something on this list.

	You need	How to check
[ ]	**Lab 0 - SCA101 Setup** passed (at least the software checks).	ChipWhisperer, numpy, matplotlib and holoviews import without errors.
[ ]	Jupyter running from your ChipWhisperer environment.	On Linux see the Linux guide Part 3; the first cell shows no `ModuleNotFoundError`.
[ ]	The course notebooks, with the `traces` folder next to the lab files.	`jupyter/courses/sca101/` contains the Explorer notebook and a `traces` folder with four `.npy` files.
[ ]	**Only for Step 7:** the Husky kit (CW313 + CW312T-SAM4S) assembled and plugged in.	Lab 0's hardware test ends with `hardware OK (disconnected again)`.

WATCH OUT

Open the Explorer notebook, not the `(HARDWARE)` or `(SIMULATED)` files. Those two are helpers that other notebooks run for you; opened on their own they do nothing useful.

GOOD TO KNOW

You cannot break anything. The recorded traces are only ever *read*. If a plot looks wrong or an error appears, use **Kernel > Restart & Run All** and you are back to the starting state.

How every edit-and-run cell works

Do this	Keys
Run the cell and move to the next one	**Shift + Enter**
Run the cell and stay on it (best for repeated experiments)	**Ctrl + Enter**
Start over from the top	**Kernel > Restart & Run All**
Insert a cell above / below (command mode)	**Esc**, then **A** / **B**

Every cell you are meant to change starts with the line `# EDIT THESE` (marked with a pencil in the notebook). Change only what is in that block, then run the cell. Everything below it just does the drawing and the maths.

Setup - open the notebook and run it

S.1 Open the Explorer

1. Start Jupyter (for Linux: `source ~/.cwvenv/bin/activate`, then `cd ~/chipwhisperer/jupyter`, then `jupyter notebook`).
2. Click **courses**, then **sca101**.
3. Click **Lab 2_1A - Instruction Power Explorer.ipynb**.

S.2 Choose the version

The first code cell has one setting you may change:

Variable	Default	What it does
`VERSION`	`'SIMULATED'`	`'SIMULATED'` = use the recorded traces only (Steps 1 to 6, no hardware). `'HARDWARE'` = also connect to the Husky so Step 7 works.

S.3 Run the two Setup cells

Press **Shift+Enter** on the `VERSION` cell, then on the cell below it (the one that starts `%matplotlib inline`).

YOU SHOULD SEE

The second cell prints a green check mark and `Ready! Loaded 4 recorded traces of 2000 samples each.` and nothing else. That cell loads the traces and defines the helper functions used by every later step.

IF IT DOESN'T

`FileNotFoundError: ... traces/lab2_1A_trace0.npy` - the `traces` folder is missing or the notebook is not in `courses/sca101` (E-1). `ModuleNotFoundError` - you are not using your ChipWhisperer environment (Lab 0).

S.4 Optional: run everything once

To see the finished notebook before you start experimenting, choose **Kernel > Restart & Run All**. It should finish in a few seconds with no red error boxes. (Step 7 prints a polite message instead of running - that is expected without hardware.)

Steps 1 to 3 - read a trace

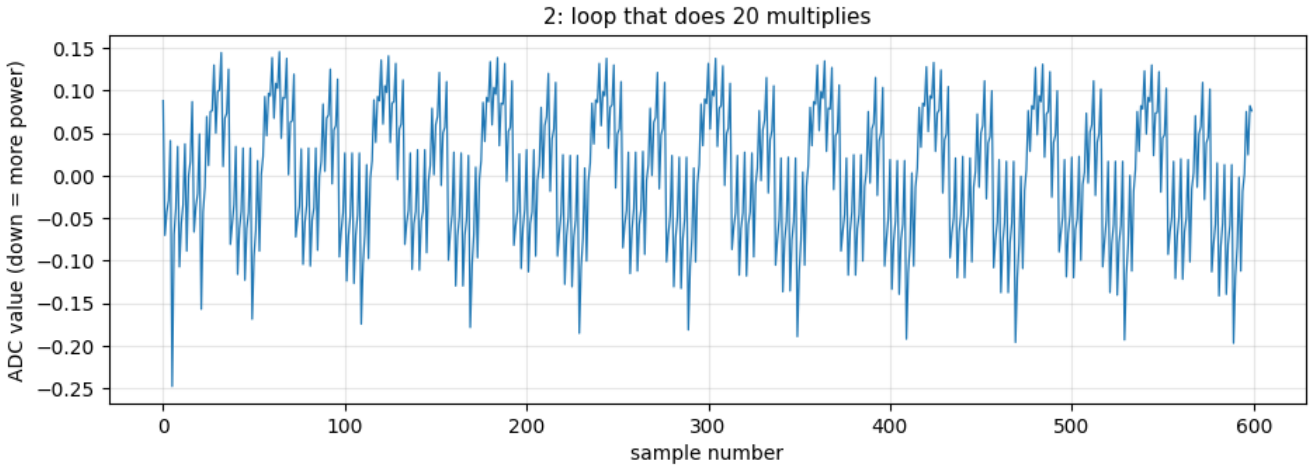
1 Look at a power trace

A **trace** is a list of **2000 numbers** - one measurement per **sample**. The x-axis is the sample number (time, in tiny steps). The y-axis is the measured value. **Careful:** ChipWhisperer measures a *voltage drop* across a resistor, so the trace is **upside down** - larger negative swings mean **more** power. `FLIP` turns it the right way up.

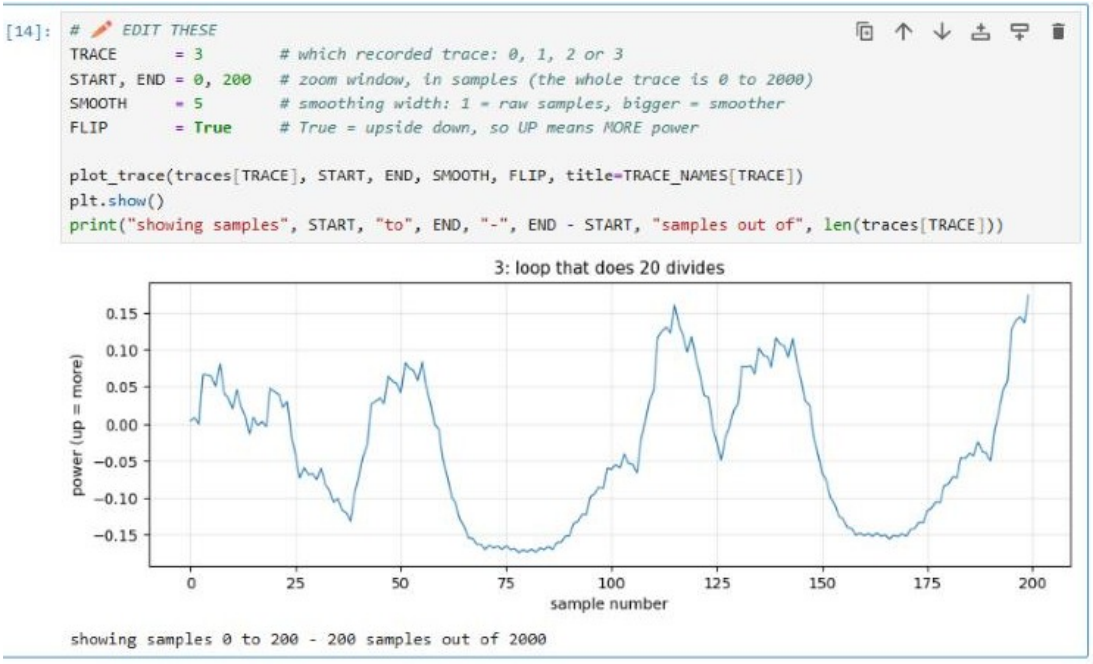
Trace	What the target was doing between the trigger signals
0	Nothing (the baseline)
1	20 multiplies, one after another
2	A loop that does the multiply 20 times
3	A loop that does a divide 20 times

Edit these:

Variable	Default	What it does
`TRACE`	`2`	Which recorded trace to show: 0, 1, 2 or 3.
`START, END`	`0, 600`	The zoom window, in samples (the whole trace is 0 to 2000).
`SMOOTH`	`1`	Smoothing width. 1 = raw samples; bigger = smoother.
`FLIP`	`False`	`True` turns the plot upside down so that UP means MORE power.



Before: the cell as shipped (TRACE = 2, window 0 to 600, raw samples, not flipped).



After: a screenshot of the real notebook running on Linux - I changed TRACE to 3, END to 200, SMOOTH to 5 and FLIP to True, then pressed Ctrl+Enter.

YOU SHOULD SEE

The plot redraws and the line under it says `showing samples 0 to 200 - 200 samples out of 2000`. In the second picture the divide loop's humps now point **up** (`FLIP`) and the little spikes are mostly gone (`SMOOTH`).

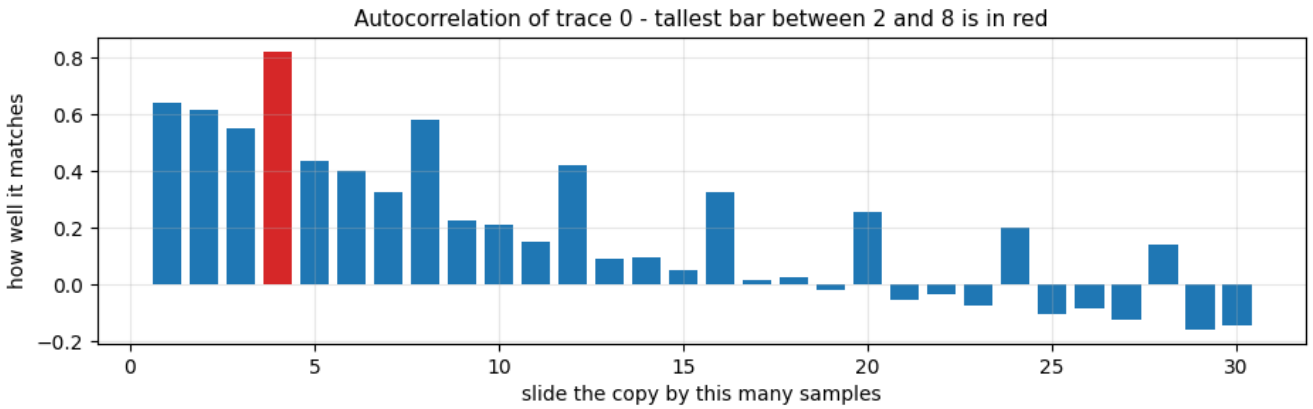
DO THIS

Try: (1) Set `START, END = 0, 100` and count the individual samples. (2) Raise `SMOOTH` to 9 - what disappears? (3) Look at all four traces. Which two look most alike? Which one has a clearly repeating shape?

2 How fast is the clock? (measure it yourself)

The ChipWhisperer takes several samples per clock cycle. **How many?** You will measure it with **autocorrelation**: compare the trace with a copy of itself slid sideways by `L` samples. If something repeats every `L` samples, the copies line up well at that `L` and the score is high.

Variable	Default	What it does
`TRACE_FOR_CLOCK`	`0`	Which trace to analyse (0 to 3).
`LAG_MIN, LAG_MAX`	`2, 8`	Search for the repeat period between these two numbers of samples.



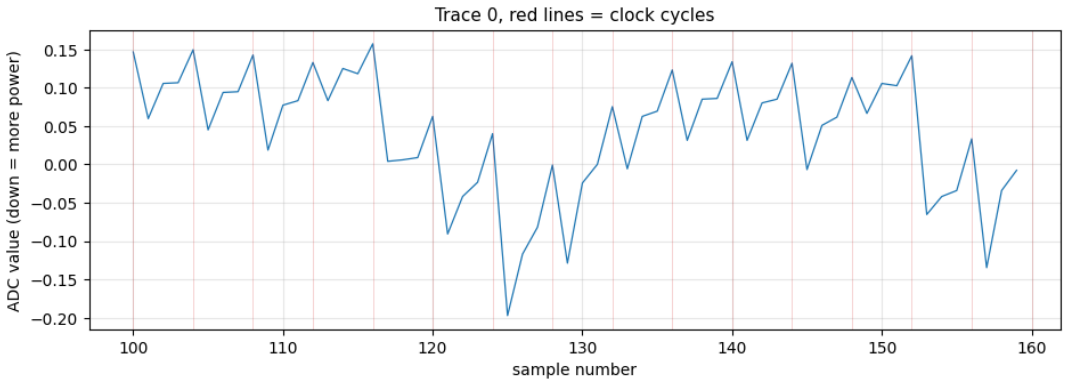
The tallest bar between lag 2 and lag 8 (red) is at 4: the trace repeats every 4 samples.

YOU SHOULD SEE

`best repeat period: 4 samples -> one clock cycle = 4 samples`. **So there are 4 samples per clock cycle.** The next cell draws a red line at every cycle - the shape repeats from line to line.

WATCH OUT

Try `LAG_MIN, LAG_MAX = 5, 12` and the winner becomes **8** - a *multiple* of the real answer (a pattern that repeats every 4 samples also repeats every 8). **Then put it back to** `2, 8` **and run the cell again.** Every later step uses this number; if you leave 8 there, all the cycle counts come out double.

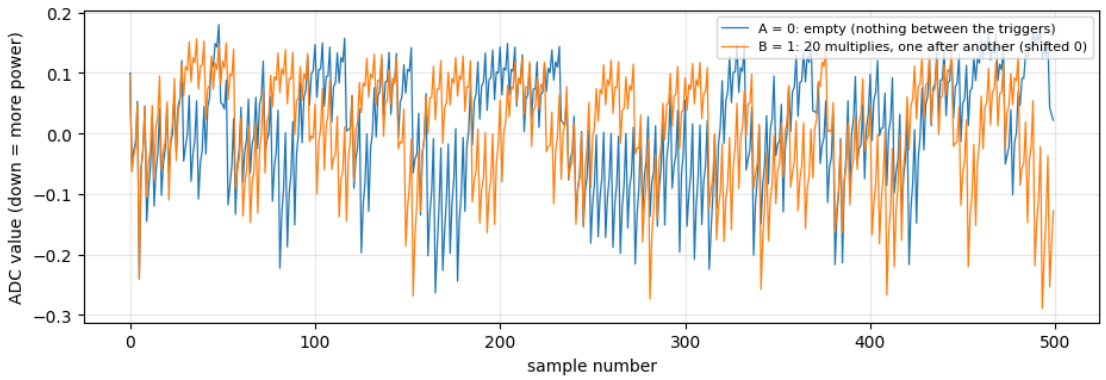


Zoom on trace 0 with red lines at every clock cycle (60 samples = 15 cycles).

3 Compare two traces

Plot any two traces on top of each other and **slide one sideways**. Does the baseline change when you add 20 multiplies?

Variable	Default	What it does
`A_TRACE, B_TRACE`	`0, 1`	Which two traces to compare.
`SHIFT_B`	`0`	Slide trace B right (+) or left (-) by this many samples.
`START, END`	`0, 500`	Zoom window.



Trace 0 (blue) against trace 1 (orange) with SHIFT_B = 0.

YOU SHOULD SEE

`average difference between the two in this window: 0.0943`, then `for scale: the wiggles in trace A are typically 0.0942 in size`. The difference is as big as the wiggles themselves, so the two traces **do not line up**. Adding 20 instructions delays everything that follows them, which is one reason a straight comparison is hard.

DO THIS

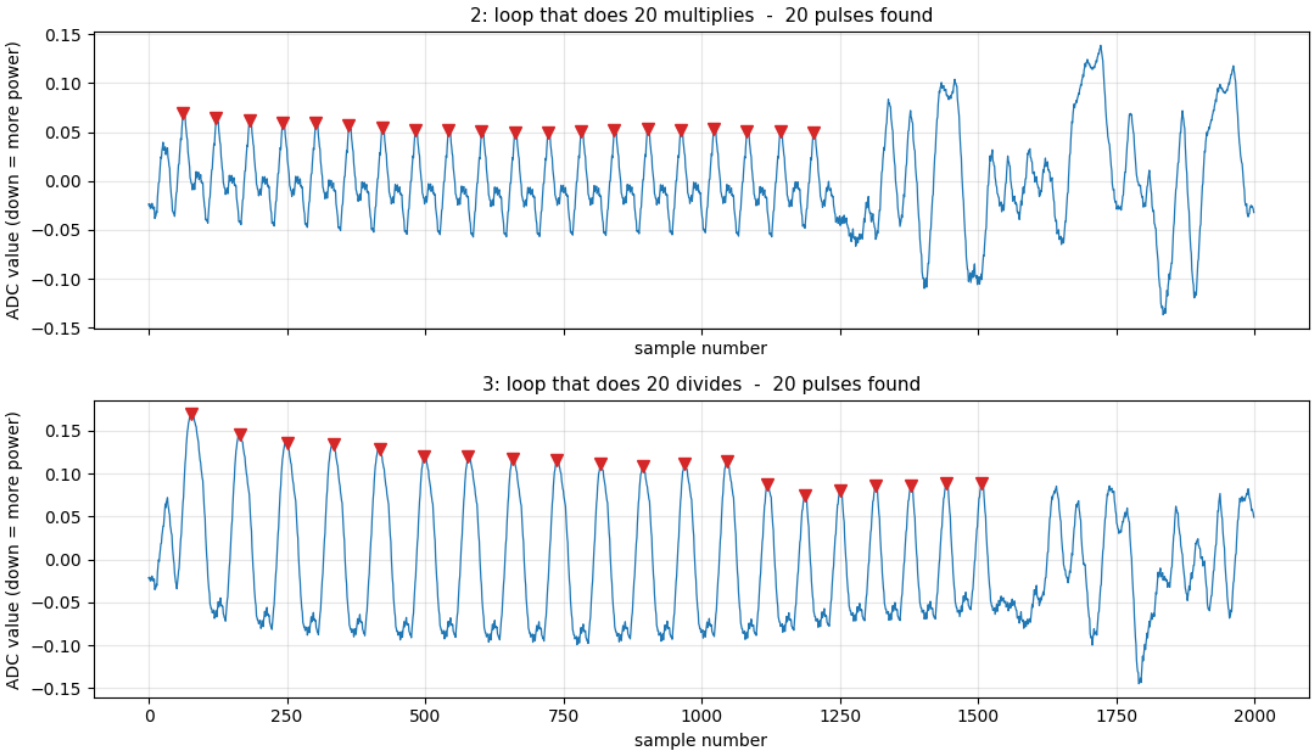
Try: (1) `0, 2` - the loop is much easier to spot than 20 loose multiplies. (2) Use `SHIFT_B` to slide the orange trace; does it ever line up with the blue one? (3) Why might 20 single instructions be hard to see? *(Each takes only a cycle or two - a handful of samples.)*

Steps 4 and 5 - find the loop

4 Find the loop

A loop runs the same few instructions again and again, so its trace is **the same shape repeated** - one **pulse** per trip round the loop. `find_pulses()` smooths the trace, finds the repeating highs, and stops when the pattern stops repeating.

Variable	Default	What it does
`SMOOTH_WIDTH`	`21`	How hard to smooth before looking for highs.
`MIN_GAP`	`30`	Highs closer together than this many samples count as one pulse.
`TOLERANCE`	`0.4`	How much the spacing may change from pulse to pulse (0.4 = 40%).
`LOOP_TRACES`	`[2, 3]`	Which traces to analyse.



Both loops with the finder's markers (red triangles): 20 pulses each.

YOU SHOULD SEE

`trace 2: 20 pulses`, spacing in samples: **60** every time (= 15 cycles). `trace 3: 20 pulses`, spacing in samples: `[88, 84, 84, 84, 80, 80, 80, 80, 80, 76, 76, 76, 72, 68, 64, 64, 64, 64, 64]` (= 22 down to 16 cycles). The multiply loop takes the same time every trip; the divide loop **gets faster as it goes**.

What we got when we changed `SMOOTH_WIDTH` (everything else at its default):

`SMOOTH_WIDTH`	3	11	21	41	101
Pulses found in trace 2 (multiply loop)	2	20	20	20	19
Pulses found in trace 3 (divide loop)	9	20	20	20	2

DO THIS

Try: (1) `SMOOTH_WIDTH = 3`: too little smoothing - noise makes fake highs. (2) `101`: too much - the pulses blur together. (3) Find the range that is right for **both** traces (11 to 41 here). (4) Set `LOOP_TRACES = [0, 1, 2, 3]`. Traces 0 and 1 have no loop, yet the finder reports a couple of 'pulses' (2 and 7 at the default setting). Look at their *spacing* lists - they are not regular. **Never trust a count without looking at the spacing.**

GOOD TO KNOW

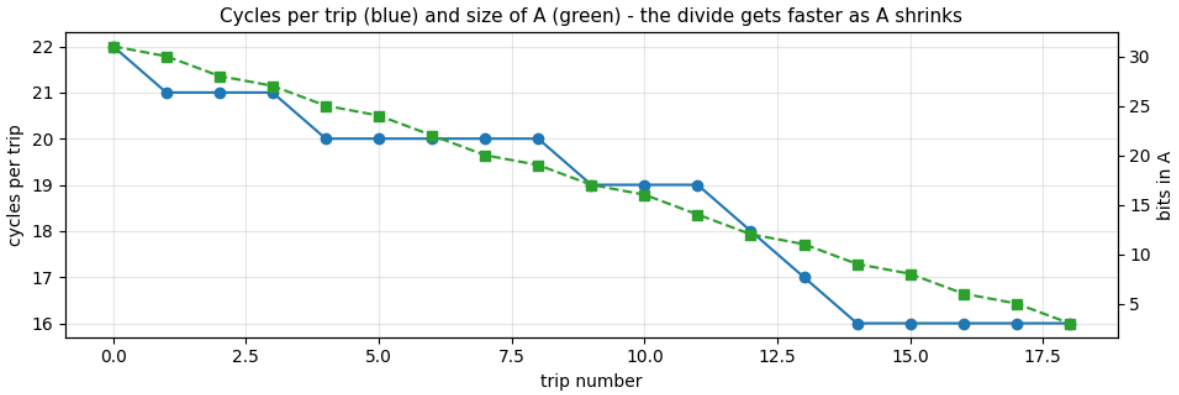
What is actually in the loop? The compiler turned `A *= 2` into a **shift** (`lsls`), not a multiply - a free optimisation - and one trip round the loop is roughly **ten instructions**, most of them loading and storing the two `volatile` variables (`A` and the counter `i`). The divide loop uses the real `sdiv` divide instruction instead. (Listing from `arm-none-eabi-gcc` 10.3.1; other compiler versions arrange things slightly differently.)

5 Why does the divide loop speed up?

Division is hard for a processor. The ARM documentation says the divide instruction takes **between 2 and 12 cycles depending on the numbers it is dividing**. The firmware did `A /= 3` twenty times starting from `A = 0x7FFFFFFF`, so after every trip `A` is a third of what it was. The notebook prints the size of `A` (in **bits**) next to the measured time of each trip.

Variable	Default	What it does
`A_START`	`0x7FFFFFFF`	The value the recorded firmware started with.
`DIVISOR`	`3`	The number it divided by.

trip	A before the divide	bits in A	measured cycles
0	`0x7fffffff`	31	22
1	`0x2aaaaaaa`	30	21
4	`0x1948b0f`	25	20
9	`0x1aa2f`	17	19
12	`0xfc8`	12	18
14	`0x1c0`	9	16
18	`0x5`	3	16



Cycles per trip (blue) and bits in A (green) fall together.

YOU SHOULD SEE

The last line prints `correlation between 'bits in A' and 'cycles per trip': r = +0.97`. Near **+1** means the two move together: **the smaller the number, the faster the divide.** The *data* changes the *timing*.

DO THIS

Try: change `A_START` and `DIVISOR` and watch the `A` and `bits` columns change. The *measured* column does not - the recording used `0x7FFFFFFF` and `3`. You can test a different start value for real in Step 7.

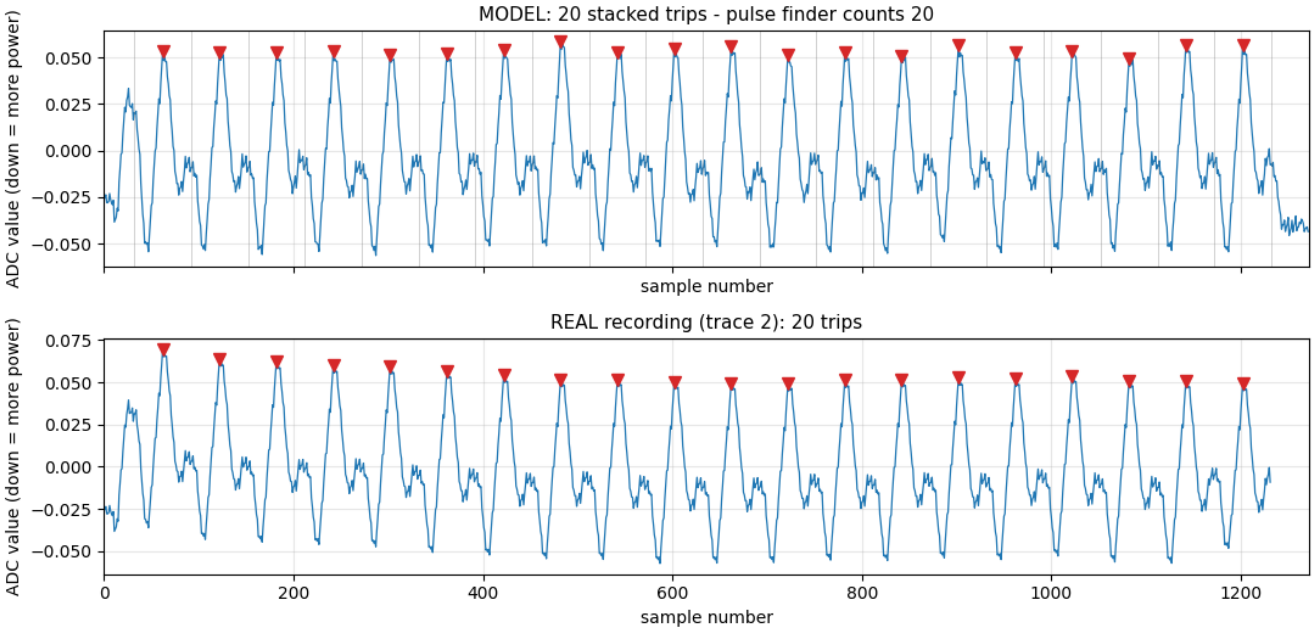
Step 6 - what if the loop ran 5 times? Or 50?

On hardware you would edit the C code, rebuild and reflash to find out. Here is a shortcut: take the **real multiply loop**, average its middle trips into one clean **template trip** (60 samples = 15 cycles, averaged from 18 real trips), **stack copies** to make a trace for any loop count, and add noise.

WATCH OUT

This is a model built from real data, not a new measurement. Real hardware has details it does not capture. Step 7 lets you check a prediction for real.

Variable	Default	What it does
`LOOP_COUNT`	`20`	How many trips round the loop to build.
`NOISE`	`0.01`	Noise added to the model: 0 = perfectly clean, 0.1 = quite noisy, 0.3 = awful.
`SEED`	`1`	Change for a different random noise pattern.



Top: the model with 20 stacked trips (finder counts 20). Bottom: the real recording.

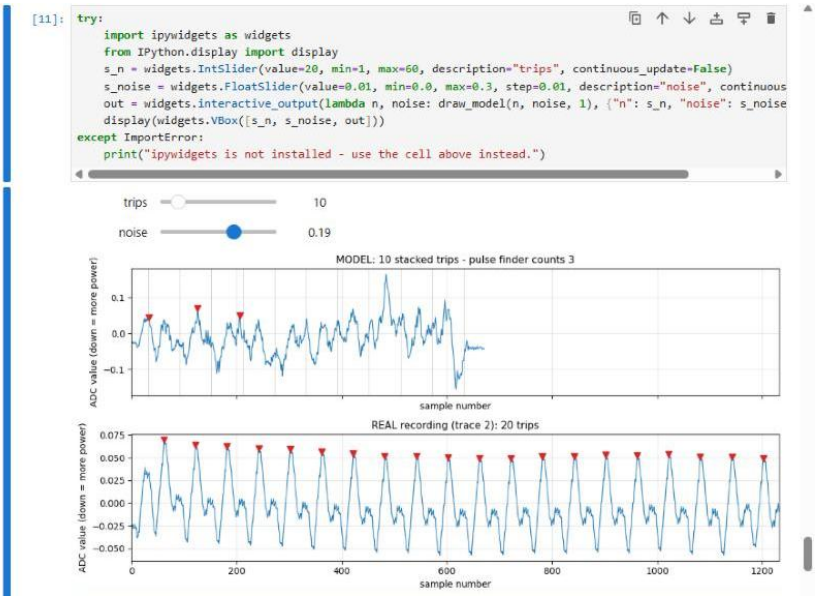
YOU SHOULD SEE

`prediction: 20 trips x 60 samples = 1200 samples = 300 clock cycles`, and the model's pulses sit at the same places as the real recording's. The notebook also prints that the real recording's own noise is only about `0.0045` - it is a very clean recording.

The pulse finder's count against noise (smoothing fixed at 21) - what we got:

NOISE	0	0.02	0.05	0.1	0.2	0.3
LOOP_COUNT = 5: pulses found	5	5	5	5	3	3
LOOP_COUNT = 20: pulses found	20	20	20	20	3	3
LOOP_COUNT = 40: pulses found	40	40	40	26	3	3

Slider version. The next cell shows the same model with two sliders (trips and noise). Drag them and the plot redraws. If the sliders do not appear, use the cell above - it does the same thing.



Sliders at 10 trips and noise 0.19: the model is so noisy the pulse finder says 3, not 10.

DO THIS

Try: (1) `LOOP_COUNT = 5`, then `50`; check the printed prediction (*trips x 60 samples*). (2) Raise `NOISE` until the count goes wrong - where does it fail? (3) Go back to Step 4 and raise `SMOOTH_WIDTH`: can you rescue the count? (4) Why can we stack copies of the **multiply** loop's trip but **not** the divide loop's? *(Step 5: its trips are all different.)*

Step 7 - your turn on the real chip (hardware only)

Now test your predictions on the real target. The cell **edits the C code that runs between the trigger signals**, rebuilds the firmware, flashes the chip, captures a fresh trace and runs the same `find_pulses()` on it. Each run takes about 10 to 20 seconds because it rebuilds and reflashes.

DO THIS

Before you start: (1) Assemble the Husky kit as in Lab 0 and plug it in. (2) In the first cell set `VERSION = 'HARDWARE'`. (3) Choose **Kernel > Restart & Run All** - the Setup cells now also connect to the Husky and flash a baseline firmware. (4) Close any other notebook that is connected to the Husky: only one program can hold it.

Variable	Default	What it does
`MY_OPERATION`	`"A *= 2;"`	The C statement inside the loop. Try `"A += 2;"`, `"A ^= 0x55;"`, `"A *= 3;"`, `"A /= 3;"`.
`MY_COUNT`	`20`	How many trips round the loop.
`MY_A_START`	`"0x2BAA"`	Starting value of `A`. For a divide try `"0x7FFFFFFF"` or `"0x100"`.

With the defaults the notebook builds this C code between the triggers:

```
volatile long int A = 0x2BAA;
volatile int i;
for (i = 0; i < 20; i++) {
    A *= 2;
}
```

YOU SHOULD SEE

The cell prints the C code, rebuilds, flashes, and shows a plot titled `YOUR capture: A *= 2; - 20 pulses found (you asked for 20)`, followed by `pulses found: 20 | asked for: 20` and a list of cycles per trip. For `A *= 2;` the recorded loop took 15 cycles per trip; your live numbers should be close but depend on the compiler version.

Experiments - predict first, then run:

Change	What do you predict?	How you will know
`MY_COUNT = 10`, then `40`	How many pulses? How long is the whole loop?	Pulse count equals `MY_COUNT`; total length is about `MY_COUNT x trip length`.
`"A += 2;"`, `"A ^= 0x55;"`, `"A *= 3;"`	Is the trip still about 15 cycles? Can you tell an add from an XOR **by timing**?	Compare the printed cycles per trip.
`"A /= 3;"` with `"0x7FFFFFFF"`	Do the trips get shorter, as in Step 4?	The printed cycles per trip should shrink.
The same divide with `"0x100"`	`A` is small from the start. Do the trips still change length?	Compare with the line above.

> **WATCH OUT**
>
> **Safe to experiment:** the notebook edits a private copy of the firmware (`firmware/mcu/simpleserial-base-lab2`) and restores a clean copy every time the Setup cell runs. Your own change never touches the original `simpleserial-base`.

Step 8 - check your understanding

Try each question first. The answers are underneath - cover them up!

#	Question	Answer
1	How many samples make one clock cycle? How did you find out?	**4.** The autocorrelation has its tallest bar at 4 samples (Step 2): the shape repeats every 4 samples, and that is the clock.
2	How many pulses did the multiply loop make, and how many cycles was each trip? And the divide loop?	**20** pulses in each. The multiply loop: **60 samples = 15 cycles** every trip. The divide loop: from about **88 samples (22 cycles)** down to about **64 samples (16 cycles)**.
3	The divide loop's pulses move closer together. Why?	The divide finishes sooner when its numbers are smaller. `A` is divided by 3 every trip, so it needs fewer bits and each trip takes fewer cycles. **The data changes the timing** - the key idea behind many side-channel attacks.
4	Why is a loop of 20 easier to spot than 20 instructions written out one after another?	A loop repeats the same shape many times, so the repetition stands out. 20 separate one-cycle instructions are a short burst of near-identical activity that blends into what is around it (and the loop adds about ten instructions of overhead per trip).
5	If the multiply loop ran 10 times instead of 20, what would you expect? What about 40?	**10 pulses** and a loop about **half as long** (about 600 samples). **40 pulses** and about **2400 samples** - if your capture window is shorter than that you only see the first part.
6	We wrote `A *= 2;` - is the chip actually using a multiplier? How would you find out?	**No.** The compiler replaced `* 2` with a *shift* (`lsls`), which is cheaper. You find out by reading the compiled code (the `.lss` file after a hardware build). Never assume the machine does what the C source looks like it does.

You are done when...

	Check
[ ]	You measured **4 samples per clock cycle** yourself (Step 2).
[ ]	You found **20 pulses** in both loops and can say how long a trip takes in cycles (Step 4).
[ ]	You can explain **why the divide loop speeds up** (Step 5).
[ ]	You predicted what a different loop count would look like (Step 6).
[ ]	**With a Husky:** you changed the C code and checked a prediction on the real chip (Step 7).

Next: Lab 2_1B uses the same idea to break a password check - the chip works a little longer for each correct character, and that shows up in the trace.

Troubleshooting

Find the message you see, read what it means, try the fix. The **Source** line says how each entry was established.

The first six steps (no hardware)

#	What you see	What it means	Fix
E-1	`FileNotFoundError: ... traces/lab2_1A_trace0.npy`	The `traces` folder is missing, or the notebook is not in `courses/sca101`. Source: Notebook source code	Check `jupyter/courses/sca101/traces/` holds four `.npy` files and that the Explorer notebook sits beside it. An old copy of the notebooks may lack them - use the course version.
E-2	`NameError: name 'traces' is not defined` (or `find_pulses`, `plot_trace`, ...)	The Setup cells have not been run in this session (for example after a kernel restart). Source: Notebook source code	**Kernel > Restart & Run All**, or run the two Setup cells again.
E-3	`RuntimeError: Run the Step 2 cell first - it measures how many samples make one clock cycle.`	A later step needs the number Step 2 measures. Source: Notebook source code	Run the Step 2 cell (the autocorrelation one), then the step you were on.
E-4	Later cycle counts are twice what the guide says	You left `LAG_MIN, LAG_MAX = 5, 12` in Step 2, so the clock measured 8 samples per cycle. Source: Reproduced	Set `LAG_MIN, LAG_MAX = 2, 8`, run the Step 2 cell again, then re-run the later cells.
E-5	The plot or numbers did not change after you edited a cell	Edits only take effect when you run the cell. Source: General Jupyter	Press **Shift+Enter** (or **Ctrl+Enter**) on the cell you edited.
E-6	No plot appears, only text such as `<Figure size 1100x340 ...>`	The Setup cell's `%matplotlib inline` did not run, or matplotlib is missing. Source: General Jupyter	Re-run the Setup cell. If it still fails: `python -m pip install matplotlib` in your ChipWhisperer environment, then restart the kernel.
E-7	The pulse count is wrong (too many, too few, or 2 pulses from a trace with no loop)	The smoothing is too light or too heavy, or the trace has no real loop. Source: Reproduced	Look at the printed **spacing** list, not just the count. Try `SMOOTH_WIDTH` between 11 and 41 (see the table in Step 4).
E-8	The sliders do not appear, or `Error displaying widget: model not found`	`ipywidgets` is missing, or the notebook was reopened without re-running the cell. Source: General Jupyter	Re-run the slider cell. If it prints `ipywidgets is not installed`: `python -m pip install ipywidgets`. The cell above the sliders does the same job.

Step 7 (hardware)

#	What you see	What it means	Fix
E-9	`Step 7 needs a ChipWhisperer. Set VERSION = 'HARDWARE' ...`	`VERSION` is still `'SIMULATED'`. Source: Notebook source code	Set `VERSION = 'HARDWARE'` in the first cell, then **Kernel > Restart & Run All**.
E-10	`RuntimeError: firmware build failed` with compiler text above it	The firmware could not be built: missing compiler, missing C library, or the notebooks are not inside the `chipwhisperer` folder. Source: Notebook source code	Run Lab 0 check 3 (`make` and `arm-none-eabi-gcc`). On Linux see the Linux guide entries L-24 to L-26.
E-11	`OSError: Could not find ChipWhisperer` or `Unable to communicate with found ChipWhisperer ...`	The Husky is not visible, or another program already holds it. Source: Source code	**Kernel > Shut Down All Kernels** (and close other lab tabs), then unplug and replug the Husky. On Linux see Linux guide 6.3.
E-12	`OSError: timeout` (SAM-BA) or `Verify FAILED` while flashing	The SAM4S target board is not the one on the CW313 (the look-alike iCE40 FPGA board), is not seated, or the Husky is stuck. Source: Husky kit bring-up	Check the board label says SAM4S, reseat it, check the 20-pin ribbon, then **physically unplug and replug the Husky** (a stuck Husky can also fail at its first FPGA upload with `usb1.USBErrorTimeout`) and run the Setup cells again.
E-13	`IOError: capture_trace timed out 3x - check trigger (CW313 JP2 = T-GPIO4) and target`	The scope never saw the trigger: wrong jumper, target not running, or no power. Source: Notebook source code	Check CW313 jumper `JP2` is on **T-GPIO4**, the target is powered over the ribbon, then run the Setup cells again.
E-14	The captured trace is almost flat	The Measure SMA cable is not connected properly. Source: Husky kit bring-up	Reseat both ends of the Measure cable (finger-tight) as in the Lab 0 assembly list.
E-15	The cell shows `[*]` for a long time	Step 7 is rebuilding and reflashing (10 to 20 s). Source: Notebook source code	Wait. If it is still busy after a minute, use **Kernel > Interrupt**, then check E-11 and E-12.

Appendix A - every variable you can edit

Step	Variable	Default	What it does
Setup	`VERSION`	`'SIMULATED'`	`'SIMULATED'` = recorded traces; `'HARDWARE'` = also connect to the Husky.
1	`TRACE`	`2`	Which trace to show (0 to 3).
1	`START, END`	`0, 600`	Zoom window in samples.
1	`SMOOTH`	`1`	Smoothing width (1 = raw).
1	`FLIP`	`False`	Turn the plot upside down so up = more power.
2	`TRACE_FOR_CLOCK`	`0`	Trace used to measure the clock.
2	`LAG_MIN, LAG_MAX`	`2, 8`	Range of repeat periods to search. **Put back to 2, 8 after experimenting.**
2	`ZOOM`, `WIDTH`	`100`, `60`	Where to look, and how many samples to show, with cycle lines.
3	`A_TRACE, B_TRACE`	`0, 1`	Which two traces to overlay.
3	`SHIFT_B`	`0`	Slide trace B sideways by this many samples.
4	`SMOOTH_WIDTH`	`21`	Smoothing used by the pulse finder.
4	`MIN_GAP`	`30`	Smallest allowed spacing between pulses.
4	`TOLERANCE`	`0.4`	Allowed change in spacing from pulse to pulse.
4	`LOOP_TRACES`	`[2, 3]`	Traces to analyse.
5	`A_START`, `DIVISOR`	`0x7FFFFFFF`, `3`	Starting value and divisor used for the table of `A`.
6	`LOOP_COUNT`	`20`	Trips in the model.
6	`NOISE`	`0.01`	Noise added to the model.
6	`SEED`	`1`	Random seed for the noise.
7	`MY_OPERATION`	`"A *= 2;"`	C statement inside the loop.
7	`MY_COUNT`	`20`	Trips round the loop on the real chip.
7	`MY_A_START`	`"0x2BAA"`	Starting value of `A` on the real chip.

Appendix B - Glossary

Term	Meaning
sample	One measurement of the power signal. A trace is a list of samples (here 2000).
clock cycle	One tick of the target's clock. The chip does its work in these steps. Here, 4 samples = 1 cycle.
trace	The recorded power signal for one run of the code between the trigger signals.
trigger	A signal from the target that tells the ChipWhisperer when to start and stop recording.
trip (iteration)	One pass round a loop. A loop of 20 makes 20 trips.
pulse	The repeating hump a loop leaves in the trace - one per trip.
autocorrelation	Comparing a signal with a slid copy of itself to find how often it repeats.
smoothing (moving average)	Replacing each sample with the average of its neighbours to reduce noise.
volatile	A C keyword that stops the compiler removing or merging accesses to a variable, so the work really happens.
`.lss` **file**	The compiler's listing of the machine code it produced. Read it to see what the chip really runs.
`sdiv`	The ARM instruction for signed divide; its time depends on the numbers.
model	A simplified stand-in for the real thing, here built by stacking real recorded trips. Useful for predicting, not a measurement.

Notice

This guide accompanies ChipWhisperer course material that references (commercial) training material on ChipWhisperer.io. You can freely execute and use the lab per the open-source license (including in your own courses if you distribute similarly), but you must maintain notice about the source: https://www.ChipWhisperer.io.

Copyright (C) NewAE Technology Inc., 2015-2020. ChipWhisperer is a trademark of NewAE Technology Inc., claimed in all jurisdictions, and registered in at least the United States of America, European Union, and Peoples Republic of China. Tutorials derived from our open-source work must be released under the associated open-source license, and notice of the source must be clearly displayed.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.